Over time, the infatuation with fixtures has dimmed, but they are still a quick and easy way to get data into your tests.

What's a Fixture?

Generically, a fixture is any known baseline state that can be made to exist at the beginning of a test. The existence of a fixed state makes it possible to write tests that make assumptions based on that particular set of data. In Rails, the term "fixture" refers to a specific mechanism baked into the framework to easily define a set of objects that will exist globally for all tests. These fixtures are defined in a set of YAML files that are automatically written to the database and converted to ActiveRecord objects at the beginning of each test run.

Each ActiveRecord model in your application can have an associated fixture file. That fixture file is named after the plural version of the model. So, if you wanted fixtures for your `Projects` and `Task` models, they would go in `spec/fixtures/projects.yml` and `spec/fixtures/tasks.yml`, respectively. (If you're using Minitest, the directory would be `test/fixtures/`.) If you use Rails generators to create your model, then a fixture file is created for you, with some boilerplate values for each attribute.

The fixture file is in YAML format, a data-description format often used as an easier-to-type alternative to XML. The details of YAML syntax are both way outside the scope of this book and largely irrelevant to fixtures. YAML contains a number of advanced features that don't concern us here.

Each entry in a fixture file starts with an identifier, with the attributes for that entry subordinate to the identifier. Here's a sample for `Project`:

```
runway:
  name: Project Runway
  due_date: 2016-12-18

book:
  name: Write the book
  due_date: 2017-04-14
```

YAML syntax is somewhat reminiscent of Python, both in the colon used to separate key/value pairs and in the use of indentation to mark the bounds of each entry. The fact that the line `book:` is outdented two spaces indicates to the YAML parser that a new entry has begun. Strings do not need to be enclosed in quotation marks (except for strings the YAML parser would find ambiguous, such as if the string value also contains a colon and a space). It doesn't hurt to add the quotation marks if you find it more readable, though.